ADR 001: CRDT collaboration through a server relay

Status

Implemented choice; formal group acceptance pending peer review. Corrected on 28 October 2026 against GitHub main at ac472ae35abfe129064e90de9e1e8f55dce19aee. This record describes that implementation, not unpushed local development. It supersedes the earlier presence-only description.

Context

The final platform must preserve edits from multiple users despite concurrency, delay and reconnection. We need a design the group can explain and test. Document content, ephemeral presence and durable storage have different lifecycles.

Operational Transformation (OT) transforms operations relative to concurrent operations. A central ordering/versioning service can fit an online editor well, but transformation correctness and history/reconnect handling must remain consistent across the chosen text operations. ShareDB supplies an OT-based backend, so the alternative is a mature library, not a request to invent transforms from scratch.

A CRDT encodes mergeable updates with identifiers and causal information. Yjs offers shared types, editor bindings and transport providers. Under its algorithm's assumptions, replicas that eventually receive the same updates converge; the application must still deliver updates, authorize users, persist state and bound resource use. CRDT metadata and compaction complicate storage and debugging. It does not eliminate all user-visible edit surprises.

Decision

Evaluation criteria, in priority order: correctness under reconnection, availability of maintained text/editor bindings, testability, local reproducibility, and operational complexity. These are qualitative criteria, not measured benchmark scores. Both Yjs and ShareDB are credible: ShareDB supports offline change synchronization too, so offline support alone does not decide the choice. Select Yjs because mergeable text updates and its editor ecosystem fit collaborative Markdown, while retaining a server for authorization and durability. Yjs is already integrated into the GitHub baseline. This is an engineering choice, not a measured claim that Yjs outperforms ShareDB. Revisit it if later requirements or experiments favour another approach.

Use a Yjs text CRDT connected through an authenticated client-server WebSocket relay. The relay is authoritative for access, admission and persistence, while CRDT merging determines text convergence. Keep presence ephemeral and separate from persistent document state.

Reject a peer-to-peer WebRTC mesh with cloud fallback for the initial product. It adds signalling, NAT traversal, relay fallback, peer lifecycle, access revocation and distributed persistence decisions before they serve an accepted requirement. A server relay is simpler to operate and observe locally, at the cost of a central availability dependency.

The GitHub baseline already merges live text through Yjs, beyond the Task 1 minimum. The browser joins a document over /ws and sends incremental Yjs updates with sequence numbers. The server checks editing permission, applies the update, persists Markdown and encoded Yjs state, and broadcasts full state with document metadata; the originating client receives an acknowledgement. Presence rosters and ping/pong demonstrate the required two-way channel. HTTP saves additionally use a conditional version check and return 409 for stale versions. That check protects snapshot saves; it is not the CRDT merge algorithm.

Consequences

- Future CRDT tests must simulate duplicate, reordered, delayed and dropped updates followed by reconnection, and assert convergence and durability.

- SQLite currently stores an encoded Yjs state alongside raw Markdown, version and update time. Keep these representations consistent when saving. State-vector/delta optimisation and a durable browser outbox are not implemented in this baseline.

- Define document authorization at both HTTP and socket boundaries. A convergent data structure does not provide access control.

- A single relay is sufficient for this milestone. Horizontal scaling needs coordinated rooms and shared durable state and is not implemented here.

- Pending client updates are held in memory, so page reload or closure can lose unsent work. A CRDT alone does not make this editor offline-first or guarantee recovery. Full-state broadcasts and CRDT metadata increase bandwidth/storage; future optimisation must preserve correctness.

AI Prompt Appendix

Actual prompts and use

These are genuine prompts from the project conversation. They are broad implementation/review requests, not a fabricated specialist research session or evidence of group approval.

1. "create repository for the beginning and do the task 1 and intro moments for the projcet" - supplied together with the Task 1 brief, including its CRDT-versus-OT and network-topology questions; used to draft the initial architecture evaluation.
2. "need to make architecture of the project(task1 and some more maybe idk just architecture). add it in marksync pdf make it clear and understandable and being 100% accurate for requirements and our project" - prompted a review of the implementation and architecture boundaries.
3. "make sure u writing about the version of the site from github because after we made repository in github and made a clone there i didnt change the version of the site in github" - constrained this correction to the published baseline rather than newer local features.

Critical commentary

The AI usefully separated transient presence, durable document state and transport, and explained that CRDT convergence does not provide authorization or message delivery. However, its earlier ADR described Yjs as future work even though the published code already used it: plausible architectural prose had become stale. The correction was checked against presence.ts, store.ts and public/app.js at the cited commit. Advice that CRDTs guarantee no lost work or that OT cannot reconnect would be misleading; delivery, unsent browser state and application recovery remain engineering responsibilities, and ShareDB supports offline synchronization. No performance benchmark or completed group acceptance is claimed. The group should review the actual code and later test convergence under reordered, duplicate and temporarily lost updates rather than treating AI recommendations as proof.

Implementation evidence and verification boundary

- app/src/presence.ts: authenticated document rooms, permissions, roster, heartbeat, Yjs update admission, broadcast and acknowledgement.

- app/src/store.ts: SQLite state, Yjs application/persistence and versioned document saves.

- app/public/app.js: Yjs client, local changes, pending updates and socket reconnection.

- app/test/app.test.ts: existing integration tests; test presence is not a claim of exhaustive distributed-systems correctness or complete Task 2 property-based testing.

- Code baseline: https://github.com/ChknNudl/Advanced-Software-Engineering/tree/ac472ae35abfe129064e90de9e1e8f55dce19aee

Primary references

- Yjs documentation: https://docs.yjs.dev/

- Yjs awareness and presence: https://docs.yjs.dev/getting-started/adding-awareness

- ShareDB introduction: https://share.github.io/sharedb/

ADR 002: TypeScript service, browser modules and SQLite

Status

Implemented choice; formal group acceptance pending peer review. Corrected on 28 October 2026 against GitHub main at ac472ae35abfe129064e90de9e1e8f55dce19aee. This record describes the published application, not unpushed local features.

Context

The marker must run the application on a commodity machine. Task 1 needs authentication, persistent document storage, Markdown rendering and genuine WebSockets. Later tasks change the parser and collaboration behavior, so transport, persistence and rendering need visible boundaries. A small team also needs a reproducible build and understandable tests.

Decision

Priorities are reproducible marker setup, understandable module boundaries, verifiable behavior and room for later changes. A framework is not currently used: this is an explicit no-framework frontend selection, not an omitted React implementation. The brief's examples should be clarified with the convenor if a framework is expected. A group decision to use React/Svelte/Vue should be recorded here and implemented through peer review rather than silently relabelling the existing UI.

Use Node.js 24 and TypeScript for the backend, Express for HTTP routing, the ws library for WebSockets, and SQLite through Node's built-in node:sqlite module. Use parameterized SQL, a versioned conditional update for save conflicts, WAL journaling and foreign keys. Store users, salted password hashes, session-token hashes and expiry, and titled documents with author references. Persist raw Markdown together with encoded Yjs state; also store projects, folders and sharing grants. The single Node process hosts HTTP and WebSocket services, so this is a modular monolith, not a microservice deployment.

Use browser-native JavaScript modules and semantic HTML/CSS for the first editor. React, Svelte or Vue could improve state management in a larger application, but this milestone has a small number of screens and no requirement for a framework. esbuild bundles the browser entry point and Yjs dependency into public/client.js. This keeps the dependency build explicit without introducing a UI framework. React offers component/state conventions but adds application structure; Svelte offers compiler-based components with another toolchain; Vue offers declarative components but would require migration. The small existing UI and reproducible local setup favour retaining vanilla JavaScript for this milestone. Revisit this decision as UI complexity grows; it is not an irreversible ban on frameworks.

Use marked for parsing and sanitize-html for output filtering, behind src/markdown.ts. Raw HTML must not become executable. Serve frontend and API from one origin to simplify session cookies and origin checks. Native WebSockets meet the required full-duplex channel without the additional application protocol supplied by Socket.IO; ws supplies server framing and control-frame handling.

Use pnpm with a committed lockfile. CI enforces ESLint, Prettier, strict backend TypeScript, Node test runner integration tests and compilation. The application works without cloud services. The repository contains root CodeRabbit configuration and a real root CI workflow after PR #15. Bot installation, actual review evidence and required branch-protection checks are separate repository concerns; configuration alone does not prove enforcement.

Consequences

- The 21 October audit moved WebSocket rooms, heartbeat and revocation into src/presence.ts, leaving HTTP handling in src/app.ts. This creates a concrete transport boundary; it does not imply distributed service isolation.

- SQLite gives a one-file local setup and transactions; concurrent writers serialize. It is appropriate for this small single-instance skeleton, not evidence that enterprise-scale throughput is solved.

- PostgreSQL would improve multi-service operations and scaling options but adds a service and setup burden. MongoDB's document model is unnecessary for this relational account/session/author model. Revisit storage when requirements justify it.

- A Node/TypeScript backend shares concepts with browser code. Go or Python are reasonable alternatives but add a language/toolchain for this team to maintain; this choice is provisional until the group confirms skills.

- Synchronous SQLite calls briefly block Node's event loop. Keep queries bounded and migrate or isolate work if measured load demands it. Password derivation uses asynchronous scrypt.

- The UI is linted JavaScript; backend TypeScript is strictly checked. Large frontend state changes may justify TypeScript and a framework later.

- Access is scoped through document authorship, project ownership and read/edit sharing grants. Projects, folders, invitation memberships and soft deletion are implemented. Signup remains open for local demonstration; production admission and operational controls need separate decisions.

- node:sqlite is version-sensitive, so Node 24 is explicitly required. Back up the data directory with the application stopped; do not copy an active database without its WAL state.

AI Prompt Appendix

Actual prompts and use

These prompts come from the project conversation. No independent team benchmark or separate targeted technology-research conversation is asserted.

1. "create repository for the beginning and do the task 1 and intro moments for the projcet" - supplied with the Task 1 brief and used to select and implement the first local stack.
2. "analyze all, we need strong project and fully fill the requirements and more for the task 1 project need to develop also need the text for group mates so they will see the project and understand so they could write documentation, share ideas release it on github and write theoretical part" - led to a broader review of maintainability, setup and documentation.
3. "make sure u writing about the version of the site from github because after we made repository in github and made a clone there i didnt change the version of the site in github" - required this correction to distinguish the published stack from later local changes.

Critical commentary

The AI's recommendation of one locally runnable service, a small relational database and separate rendering/transport modules made the skeleton easier to inspect. Its earlier assumptions were nevertheless unreliable: setup advice assumed the assistant's executable PATH matched the user's terminal, and documentation later claimed no bundler and an unrestricted shared workspace even though esbuild and access grants were implemented. The corrected record was compared with package.json, the root CI workflow and the HTTP, access and storage modules. SQLite and scrypt are useful choices, not proof of enterprise readiness: measured capacity, recovery, TLS and operational controls still matter. No framework or database benchmark is claimed. Human review and a clean-machine rehearsal remain necessary to validate the recommendation.

Implementation evidence and local execution

- app/package.json: Node 24, pnpm 11.25.0, esbuild, strict backend TypeScript, lint/format/test/build scripts. Browser JavaScript is linted; do not describe it as fully TypeScript-checked.

- app/src/app.ts and access.ts/access-routes.ts: HTTP sessions, origin checks, permissions and document routes.

- app/src/store.ts and markdown.ts: SQLite persistence and filtered Markdown rendering.

- app/src/presence.ts: ws transport and room lifecycle, distinct from database storage and browser rendering.

- .github/workflows/ci.yml: PR-to-main checks, installed from the committed lockfile. This file alone does not establish branch protection.

- Local startup from app/: pnpm install --frozen-lockfile, then pnpm dev; open http://localhost:3000. Windows PowerShell can use pnpm.cmd to avoid pnpm.ps1 execution-policy restrictions. SQLite defaults to data/marksync.sqlite. Docker Compose is an alternative with a persistent volume.

- Hosting requires a Node process, WebSocket support and durable database storage; GitHub Pages alone cannot run this application. Cloud deployment is not a Task 1 requirement.

- Code baseline: https://github.com/ChknNudl/Advanced-Software-Engineering/tree/ac472ae35abfe129064e90de9e1e8f55dce19aee

Primary references

- Node SQLite API: https://nodejs.org/api/sqlite.html

- ws implementation and examples: https://github.com/websockets/ws

- Marked security guidance: https://marked.js.org/#usage

- CodeRabbit configuration: https://docs.coderabbit.ai/reference/configuration